

# Python Fundamentals for AI Engineering

## References:

<https://www.learnpython.org/>

<https://www.youtube.com/watch?v=L5sZ6WgOnj0&t=8208s&pp=ygUScHI0aG9uIGFudWogYmhhaXIh>

<https://www.w3schools.com/python/default.asp>

## What is a Virtual Environment?

A **virtual environment** creates an isolated Python environment for a project.

Without virtual environments, installing a package for one project can affect another project:

```
Project A
├── Python
└── package v1

Project B
├── Python
└── package v2
```

Each project gets its own dependencies, making projects easier to reproduce and maintain.

Python provides this through the built-in `venv` module:

```
python -m venv .venv
```

Activate it:

```
# macOS / Linuxsource .venv/bin/activate# Windows
```

```
.venv\Scripts\activate
```

## What is `uv` ?

`uv` is a modern, extremely fast Python package and project manager written in Rust.

It can handle much more than just virtual environments:

- Python installation and version management
- Virtual environments
- Package installation
- Dependency management
- Lock files
- Project management

Instead of manually doing:

```
python -m venv .venv  
pip install ...
```

with `uv`, you can use:

```
uv init my-project  
cd my-project  
uv add requests
```

This creates a reproducible project structure with dependency information and a lock file.

## Recommended approach

For modern Python projects, we'll use `uv` to manage our Python environment and dependencies, while the underlying concept of an isolated `venv` remains important to understand.

# 1. Python Basics

## 1.1 Variables

Python variables do not require an explicit type declaration.

```
name = "Anuj"  
age = 28  
price = 99.5  
is_active = True
```

A variable is a **name bound to an object**.

```
x = 10  
x = "hello"    # valid
```

| Python                                  | Java                                      |
|-----------------------------------------|-------------------------------------------|
| <pre>x = 10</pre>                       | <pre>int x = 10;</pre>                    |
| <pre>x = "hello"</pre>                  | <pre>String x = "hello";</pre>            |
| Variable can be rebound to another type | Variable has a declared compile-time type |

## Check the type

```
type(x)
```

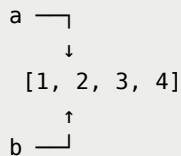
## Aliasing: two names, one object

This is the single biggest mental-model shift coming from Java's primitives-vs-objects split.

```
a = [1, 2, 3]  
b = a  
  
b.append(4)
```

```
print(a)    # [1, 2, 3, 4]
```

There is **one list object**; `a` and `b` are both names pointing at it. Mutating through `b` is visible through `a`. `=` never copies an object — it just binds a name to it.



## 1.2 Basic Data Types

```
name = "Anuj"           # str
age = 28                 # int
price = 99.5             # float
is_active = True         # bool
value = None             # NoneType
```

⚠ **None is not False, 0, or ""**. It specifically means “no value.” Check for it with `is None`, not `==`:

```
value = 0
value is None    # False – 0 is a real value
not value        # True  – 0 is falsy
```

`is None` asks “is there no value at all?”; `not value` asks “is this falsy?” These are different questions — mixing them up is a common bug once `0` or `""` are valid inputs.

Common collection types:

```
numbers = [1, 2, 3]      # list
point = (10, 20)         # tuple
```

```
unique = {1, 2, 3}           # set
user = {"name": "Anuj"}     # dict
```

## Mutable vs immutable

**Mutable:** can be changed after creation.

```
numbers = [1, 2, 3]
numbers.append(4)
```

**Immutable:** cannot be changed in place.

```
name = "Anuj"
```

Common immutable types: `int`, `float`, `bool`, `str`, `tuple`.

Common mutable types: `list`, `dict`, `set`.

## 1.3 Strings

```
name = "Anuj"
message = f"Hello{name}"
```

Useful operations:

```
name.upper()
name.lower()
name.strip()
name.split(",")
```

Strings support indexing:

```
name[0]
name[-1]
```

Slicing:

```
name[0:3]
```

## 1.4 Lists

```
numbers = [1, 2, 3]

numbers.append(4)
numbers.remove(2)

print(numbers[0])
print(len(numbers))
```

Useful patterns:

```
first = numbers[0]
last = numbers[-1]
```

List comprehension:

```
squares = [x * x for x in numbers]
```

With a condition:

```
even = [x for x in numbers if x % 2 == 0]
```

List comprehensions are common in Python code. Learn to **read them comfortably**.

## 1.5 Tuples

```
point = (10, 20)
```

Useful for fixed collections and returning multiple values.

```
def get_user():  
    return "Anuj", 28  
  
name, age = get_user()
```

This is called **unpacking**.

## 1.6 Dictionaries

Dictionaries are extremely important in Python and AI code.

```
user = {  
    "name": "Anuj",  
    "age": 28  
}
```

Access:

```
user["name"]
```

Safer access:

```
user.get("name")
```

Add/update:

```
user["city"] = "Gurgaon"
```

Iterate:

```
for key, value in user.items():  
    print(key, value)
```

Common AI code contains dictionaries everywhere:

```
config = {  
    "temperature": 0.7,  
    "max_tokens": 1000  
}
```

## 1.7 Sets

A set stores unique values.

```
tags = {"python", "ai", "python"}  
  
print(tags)
```

Useful for membership and removing duplicates.

```
if "ai" in tags:  
    ...
```

## 2. Conditions and Loops

### 2.1 `if`, `elif`, `else`

```
age = 20  
  
if age >= 18:  
    print("Adult")  
elif age >= 13:  
    print("Teenager")  
else:  
    print("Child")
```

Python uses **indentation** to define blocks.

No `{}` are required.

## Truthy and falsy values

The following are commonly falsy:

```
False
None
0
""
[]
{}
set()
```

Therefore:

```
if users:
    print("Users exist")
```

is common Python.

---

## 2.2 `for` loops

```
for number in numbers:
    print(number)
```

With `range()` :

```
for i in range(5):
    print(i)
```

`range(5)` produces `0, 1, 2, 3, 4` .

With index:

```
for index, value in enumerate(numbers):  
    print(index, value)
```

This is preferred over manually maintaining a counter.

---

## 2.3 while

```
while condition:  
    ...
```

Use it when the number of iterations is not known beforehand.

---

## 2.4 break and continue

```
for item in items:  
    if item is None:  
        continue  
  
    if item == "stop":  
        break
```

# 3. Functions

```
def add(a, b):  
    return a + b
```

Call:

```
result = add(2, 3)
```

## Type hints

```
def add(a: int, b: int) -> int:
    return a + b
```

Type hints describe intended types but generally **do not enforce runtime validation**.

This distinction becomes important with Pydantic.

## Arguments: mutate vs. reassign

Python is neither “pass-by-value” nor “pass-by-reference” in the Java sense. The accurate model: **object references are passed by value**.

```
def add_item(items):
    items.append("AI")           # mutates the shared object

items = ["Python"]
add_item(items)
print(items)                   # ["Python", "AI"]
```

```
def change_value(x):
    x = 100                     # rebinds the local name only

x = 10
change_value(x)
print(x)                       # 10 – unchanged
```

Mutating the object a reference points to is visible outside the function. Reassigning the local name inside the function is not — it’s the same aliasing model from Section 1.1, just applied to function calls. Worth demoing live.

## 3.1 Default arguments

```
def greet(name, greeting="Hello"):
```

```
return f"{greeting}{name}"
```

```
greet("Anuj")  
greet("Anuj", "Hi")
```

⚠️ **The mutable default argument trap.** Default values are created **once**, when the function is defined — not on every call:

```
def add_user(name, users=[]):    # DON'T  
    users.append(name)  
    return users
```

Calling `add_user("Anuj")` then `add_user("Rahul")` silently accumulates into the *same* list across calls. Fix by defaulting to `None` and creating fresh state inside:

```
def add_user(name, users=None):    # DO  
    if users is None:  
        users = []  
    users.append(name)  
    return users
```

This `param=None` + `if param is None` pattern is everywhere in real Python code.

## 3.2 Keyword arguments

```
create_user(  
    name="Anuj",  
    age=28  
)
```

Python allows arguments to be passed by name.

This is extremely common in AI libraries.

**Reading library signatures: the `*` marker.** You'll see signatures like:

```
def invoke(input, config=None, *, stop=None, **kwargs):  
    ...
```

Everything after a bare `*` must be passed by keyword — `invoke(x, stop=["END"])`, never `invoke(x, ["END"])`. Recognize this pattern; you don't need to design APIs with it yet.

**No Java-style overloading.** Python doesn't dispatch on parameter types or count:

```
def process(value): ...  
def process(value, count): ... # replaces the first definition — doesn't overload it
```

The common Python fix is a default argument ( `def process(value, count=None)` ), not multiple signatures.

### 3.3 `args`

Collect extra positional arguments:

```
def add_all(*args):  
    return sum(args)  
  
add_all(1, 2, 3, 4)
```

Inside the function, `args` is a tuple.

### 3.4 `*kwargs`

Collect extra keyword arguments:

```
def configure(**kwargs):  
    print(kwargs)
```

```
configure(  
    temperature=0.7,  
    timeout=30  
)
```

Inside the function, `kwargs` is a dictionary.

Very common in framework and library code.

## Unpacking

```
config = {  
    "temperature": 0.7,  
    "timeout": 30  
}  
  
model.invoke(messages, **config)
```

- `*config` expands the dictionary into keyword arguments.

The same idea works for lists with a single `*`:

```
numbers = [1, 2, 3]  
print(*numbers)    # same as print(1, 2, 3)
```

## Lambda

```
square = lambda x: x * x
```

Same as:

```
def square(x):  
    return x * x
```

You don't need to write elaborate lambdas — just read them comfortably:

```
users.sort(key=lambda user: user["age"]) # sort by age
```

## 4. `==` vs `is`

This is an important Python distinction.

```
a = [1, 2]
b = [1, 2]

a == b    # True: same value
a is b    # False: different objects
```

- `==` → equality
- `is` → object identity

Use `is` commonly with `None`:

```
if value is None:
    ...
```

Do not generally use `is` to compare ordinary values.

## 5. Functions Are Objects

Python functions can be stored in variables, passed to other functions, and returned.

```
def greet(name):
    return f"Hello{name}"

fn = greet
fn("Anuj")
```

This is important for understanding **decorators, callbacks, tools, and framework APIs**.

---

## 6. Exceptions

Python uses exceptions for error handling.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

With `else` and `finally`:

```
try:
    result = operation()
except ValueError:
    handle_error()
else:
    process(result)
finally:
    cleanup()
```

### Raising exceptions

```
if age < 0:
    raise ValueError("Age cannot be negative")
```

### Custom exceptions

```
class UserNotFoundError(Exception):
    pass
```

You will encounter exceptions constantly in APIs, SDKs, databases, and AI applications.

---

## 7. Imports and Modules

Import a module:

```
import math
math.sqrt(16)
```

Import specific items:

```
from pathlib import Path
```

Common structure:

```
project/
├── main.py
├── services/
│   ├── __init__.py
│   └── user_service.py
└── models/
    └── user.py
```

Python files are modules. Folders can form packages.

---

## 8. Virtual Environments and Packages

Create a virtual environment:

```
python -m venv .venv
```

Activate it and install packages:

```
pip install fastapi
```

```
pip install pydantic
```

A project commonly records dependencies in:

```
requirements.txt
```

or modern Python project configuration such as:

```
pyproject.toml
```

You don't need to become a Python packaging expert for this course.

---

## 9. OOP in Python

### 9.1 Class and Object

```
class User:
    name: str
    age: int

user1 = User()
user2 = User()
```

- `User` → class
  - `user1`, `user2` → objects / instances
- 

### 9.2 `__init__`

```
class User:

    def __init__(self, name, age):
```

```
self.name = name
self.age = age
```

```
user = User("Anuj", 28)
```

`__init__` initializes the object's state.

## 9.3 `self`

```
class User:

    def greet(self):
        print(f"Hello{self.name}")
```

`self` refers to the current object.

Conceptually:

```
user.greet()
```

is similar to:

```
User.greet(user)
```

`self` is a convention, not a keyword.

| Python                                  | Java                          |
|-----------------------------------------|-------------------------------|
| <code>self.name</code>                  | <code>this.name</code>        |
| <code>def greet(self)</code>            | <code>void greet()</code>     |
| <code>self</code> is explicitly written | <code>this</code> is implicit |

## 9.4 Instance Attributes

```
class User:

    def __init__(self, name):
        self.name = name
```

Each instance has its own state.

```
user1 = User("Anuj")
user2 = User("Rahul")
```

Conceptually:

```
user1 → name = "Anuj"
user2 → name = "Rahul"
```

Python objects can also be dynamically extended:

```
user1.age = 28
```

---

## 9.5 Methods

```
class User:

    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Hello{self.name}"
```

```
user.greet()
```

An instance method receives the current object as `self`.

---

## 9.6 Class Attributes

```
class User:
    species = "human"

    def __init__(self, name):
        self.name = name
```

`species` belongs to the class; `name` belongs to each instance.

Avoid mutable class attributes when you want per-object state:

```
# Avoid
class User:
    permissions = []

# Prefer
class User:
    def __init__(self):
        self.permissions = []
```

## 9.7 @classmethod

```
class User:

    def __init__(self, name):
        self.name = name

    @classmethod
    def from_dict(cls, data):
        return cls(data["name"])
```

Called using:

```
user = User.from_dict({"name": "Anuj"})
```

`cls` refers to the class.

Common use: **alternative constructors / factory methods**.

---

## 9.8 `@staticmethod`

```
class MathUtils:

    @staticmethod
    def add(a, b):
        return a + b
```

It receives neither `self` nor `cls`.

Think of it as a function placed inside a class namespace.

---

## 9.9 `@property`

```
class User:

    def __init__(self, first, last):
        self.first = first
        self.last = last

    @property
    def full_name(self):
        return f"{self.first}{self.last}"
```

```
user.full_name          # not user.full_name()
```

The decorator makes a method readable like a plain attribute — loosely like a Java getter, but called without parentheses. You'll see this in libraries that expose

computed values without breaking the external `object.attribute` access pattern.

---

## 9.10 Naming conventions for “private”

Python has no `private` / `protected` access modifiers — only conventions, enforced by nobody:

```
class User:
    def __init__(self, name):
        self._name = name    # "internal use" – a convention, not enforced
```

A single leading underscore (`_name`) signals “internal, don’t touch this from outside” and Python trusts you to respect it. A double leading underscore (`__name`) triggers **name mangling** (Python renames it internally to avoid subclass clashes) — know it exists, but a single underscore is the idiomatic default, not `__name`.

---

## 10. Inheritance

```
class Animal:

    def speak(self):
        print("Some sound")

class Dog(Animal):

    def speak(self):
        print("Woof")
```

```
dog = Dog()
dog.speak()
```

Python supports inheritance similarly to Java.

| Python                          | Java                                  |
|---------------------------------|---------------------------------------|
| <code>class Dog(Animal):</code> | <code>class Dog extends Animal</code> |
| <code>super()</code>            | <code>super</code>                    |
| Method overriding               | Method overriding                     |

## 10.1 `super()`

```
class Animal:

    def __init__(self, name):
        self.name = name

class Dog(Animal):

    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
```

Use `super()` to access parent-class behavior.

# 11. Composition

Composition is extremely important in AI engineering.

```
class Tool:
    content: str
    name: str
    description: str

class Agent:

    def __init__(self, model, tools):
        self.model = model
```

```
self.tools = tools

tool = Tool()
agent = Agent(model="GPT-5.1", tool)
```

An `Agent` has a model and tools.

```
Agent
├── model
├── tools
└── ...
```

AI systems are commonly built by composing objects:

```
Application
├── LLM
├── Retriever
├── Vector Store
├── Tools
└── Memory
```

You will see this pattern frequently in LangChain, LangGraph, SDKs, and application code.

## 12. Duck Typing

Python often cares about **behavior** rather than an explicit inheritance relationship.

```
class Dog:
    def speak(self):
        return "Woof"

class Cat:
    def speak(self):
        return "Meow"
```

```
def make_sound(animal):  
    return animal.speak()
```

Both work:

```
make_sound(Dog())  
make_sound(Cat())
```

The function doesn't require a specific parent class. It simply expects the object to provide `speak()`.

| If an object behaves like what you need, you can often use it.

This pattern is common in Python libraries and AI frameworks.

## 13. Dunder / Special Methods

Methods surrounded by double underscores are commonly called **dunder methods**.

Examples:

```
__init__  
__str__  
__repr__  
__len__  
__iter__
```

Python or a framework invokes these automatically in particular situations.

```
__str__
```

Human-readable representation:

```
class User:

    def __str__(self):
        return self.name
```

`__repr__`

Developer/debug representation:

```
class User:

    def __repr__(self):
        return f"User(name={self.name!r})"
```

## 14. Dataclasses

Dataclasses reduce boilerplate for data-oriented classes.

```
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int
```

Now:

```
user = User("Anuj", 28)
```

Python generates common methods such as `__init__` and a useful representation.

### Dataclass vs Pydantic

| Dataclass                    | Pydantic                                         |
|------------------------------|--------------------------------------------------|
| Convenient data container    | Data container + validation                      |
| Mainly Python-side structure | Strong runtime validation                        |
| Standard library             | External package                                 |
| Type hints                   | Type hints + validation + serialization + schema |

Pydantic becomes especially important at API and AI-system boundaries.

## 15. Decorators

Decorators are extremely important for FastAPI and AI libraries.

```
@decorator
def hello():
    pass
```

Conceptually:

```
hello = decorator(hello)
```

Example:

```
@app.get("/users")
def get_users():
    ...
```

The decorator modifies/registers behavior around the function.

## 16. Context Managers

You will encounter the `with` statement frequently.

```
with open("data.txt") as file:
    data = file.read()
```

The context manager handles setup and cleanup.

Common examples include:

```
with open(...):  
    ...  
  
with database_connection():  
    ...  
  
with client:  
    ...
```

## 17. Iterables and Generators

You will encounter these in Python libraries.

An iterable can be used in a loop:

```
for item in items:  
    ...
```

A generator can produce values lazily:

```
def numbers():  
    yield 1  
    yield 2  
    yield 3
```

```
for number in numbers():  
    print(number)
```

You mainly need to understand that `yield` produces values lazily rather than returning everything at once.

# 18. Async Python

This is **important for FastAPI and AI applications**.

Synchronous:

```
def get_data():  
    ...
```

Asynchronous:

```
async def get_data():  
    ...
```

Calling an async function produces a coroutine:

```
result = get_data()
```

To execute/await it inside async code:

```
result = await get_data()
```

Typical FastAPI code:

```
@app.get("/users")  
async def get_users():  
    users = await fetch_users()  
    return users
```

## Important mental model

`async` / `await` is primarily about handling operations that spend time **waiting**, such as:

- HTTP requests
- database operations

- LLM API calls
- file/network I/O

It does not automatically make CPU-heavy work faster.

⚠ **Blocking code inside `async def` still blocks.** `async` doesn't make a blocking call asynchronous by magic:

```
@app.get("/data")
async def get_data():
    time.sleep(10)      # blocks the event loop – defeats the purpose
    return {"ok": True}
```

If a library gives you an async version, use it ( `await async_operation()` ). This is a real, common FastAPI bug — more relevant day-to-day than event-loop internals.

**Naming convention to recognize:** many AI libraries pair a sync and async version of the same method, e.g. `model.invoke(...)` (sync) vs. `await model.ainvoke(...)` (async). The `a` prefix is a strong hint, not a hard rule — check the docs.

## 19. Type Hints You Will See Everywhere

Basic:

```
name: str
age: int
score: float
active: bool
```

Collections:

```
names: list[str]
scores: list[float]
```

```
users: dict[str, int]
```

Optional values:

```
name: str | None
```

Union:

```
value: int | str
```

Any:

```
data: Any
```

`Any` essentially says:

“Don’t enforce a specific static type here.”

## Type hints vs Pydantic

```
class User:  
    age: int
```

is primarily a type annotation.

```
from pydantic import BaseModel  
  
class User(BaseModel):  
    age: int
```

adds runtime validation and data parsing behavior.